

model/d2q.py (Q&A)

Q: What is the purpose of model/d2q.py? **A:** It implements the D2Q baseline model, which predicts a scalar “quantile” score (after sigmoid) from features. The accompanying run script (run_d2q.py) maps watch-time labels into bucket-specific quantile space during training and maps predictions back to watch-time values during evaluation.

Q: What are the inputs and outputs of D2Q.forward? **A:** Inputs are the standard feature dictionary `x_dict`; outputs are a scalar per example (a sigmoid-activated value) intended to represent a normalized quantile within a duration bucket.

Q: How are features encoded? **A:** It uses embeddings for sparse and sequence features and linear transforms for continuous features, concatenates them, and feeds the result into `MultiLayerPerceptronD2Q` (a Swish-based MLP).

Q: Why does D2Q need duration buckets? **A:** D2Q is designed to model the conditional distribution of play time given video duration by first bucketing duration. The mapping from watch time to quantile is bucket-specific (precomputed by preprocessing), and the model predicts in that normalized space.

Q: Code example: forward pass shape (schematic)? **A:**

```
q = model(features) # q: [batch] after sigmoid
```

Q: Full source code? **A:**

```
import torch

from model.layers import MultiLayerPerceptronD2Q

class D2Q(torch.nn.Module):

    def __init__(self, description, embed_dim, mlp_dims, dropout):
        super().__init__()
        self.features = {name: (size, type) for name, size, type in description if (type
            in ["ctn", 'seq', 'spr'])}
        self.build(embed_dim, mlp_dims, dropout)

    def build(self, embed_dim, mlp_dims, dropout):
        self.emb_layer = torch.nn.ModuleDict()
        self.ctn_emb_layer = torch.nn.ParameterDict()
        self.ctn_linear_layer = torch.nn.ModuleDict()
        embed_output_dim = 0
```

```

for name, (size, type) in self.features.items():
    if type == 'spr':
        self.emb_layer[name] = torch.nn.Embedding(size, embed_dim)
        embed_output_dim += embed_dim
    elif type == 'ctn':
        self.ctn_linear_layer[name] = torch.nn.Linear(1, 1, bias=False)
        embed_output_dim += 1
    elif type == 'seq':
        self.emb_layer[name] = torch.nn.Embedding(size, embed_dim)
        embed_output_dim += embed_dim
    else:
        raise ValueError('unkown_{}_feature_{}_type:{}'.format(type))
self.mlp = MultiLayerPerceptronD2Q(embed_output_dim, mlp_dims, dropout)
return

# def init(self):
# for param in self.parameters():
# torch.nn.init.uniform_(param, -0.01, 0.01)

def forward(self, x_dict):
    linears = []
    embs = []
    for name, (_, type) in self.features.items():
        x = x_dict[name]
        if type == 'spr':
            embs.append(self.emb_layer[name](x).squeeze(1))
        elif type == 'ctn':
            linears.append(self.ctn_linear_layer[name](x))
        elif type == 'seq':
            seq_emb = self.emb_layer[name](x)
            seq_mask = torch.unsqueeze(x_dict["{}_mask".format(name)], dim=2)
            embs.append(torch.sum(seq_emb * seq_mask, dim=1) / torch.sum(seq_mask, dim=1))
        else:
            raise ValueError('unkwon_{}_feature:{}'.format(name))
    emb = torch.concat(embs + linears, dim=1)
    res = self.mlp(emb)
    res = res.squeeze(1)
    return torch.sigmoid(res)

```